



REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

EJERCICIO 2: LA FALACIA DE LOS RECURSOS Y EL CONO DE INCERTIDUMBRE

Profesor:

Mg. Félix Márquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Ciudad Guayana, septiembre de 2026

Introducción

Uno de los errores más persistentes en la gestión de proyectos de software es asumir una relación lineal entre el esfuerzo, los recursos humanos y el tiempo de calendario. La pretensión de tratar el desarrollo de software como una línea de ensamblaje industrial conduce a decisiones ejecutivas catastróficas cuando un proyecto enfrenta retrasos. La ecuación simplista $\text{Tiempo} = \text{Esfuerzo} / \text{Recursos}$ ignora la naturaleza no lineal de la complejidad cognitiva, la comunicación humana y la integración de sistemas.

El presente ensayo analiza el Ejercicio 2 de la Unidad III, en el cual el Director de una empresa ordena duplicar el equipo de desarrollo (de 5 a 10 programadores) faltando solo 2 meses para la fecha límite de un proyecto crítico en retraso. A través del prisma de la Ley de Brooks (1975) y la teoría del Cono de Incertidumbre (McConnell, 2004), se demuestra matemática y conceptualmente por qué esta medida agravará el retraso. Asimismo, se formulan tres factores de sobrecarga operativa y se proponen dos alternativas de gestión (predictiva y ágil/híbrida) para abordar la crisis manteniendo la validez técnica del proyecto.

1. Análisis Matemático y Conceptual: La Falacia de la Relación Lineal

1.1. Demostración Matemática y Colapso de la Ecuación Tiempo = Esfuerzo / Recursos

La ecuación lineal Tiempo = Esfuerzo / Recursos parte del supuesto falso de que las tareas de desarrollo son absolutamente independientes y divisibles. Si un proyecto requiere E personas-mes de esfuerzo y se asignan N recursos, la fórmula predice $T = E / N$ meses. Sin embargo, en la ingeniería de software, el trabajo no es perfectamente divisible porque existen dependencias lógicas rígidas y necesidades constantes de comunicación entre los integrantes.

Desde la teoría de grafos y redes de comunicación, el número de canales de comunicación interpersonales en un equipo de tamaño N se calcula mediante la combinación de dos elementos en un conjunto de tamaño N :

$$\text{Canales de Comunicación} = C(N) = (N * (N - 1)) / 2$$

Evaluando el impacto directo de la decisión directiva:

- **Equipo original ($N = 5$):** $C(5) = (5 * 4) / 2 = 10$ canales de comunicación.
- **Equipo duplicado ($N = 10$):** $C(10) = (10 * 9) / 2 = 45$ canales de comunicación.

Al duplicar el número de programadores (un incremento del 100%), la sobrecarga de comunicación escala de 10 a 45 canales, lo que representa un aumento del 350% en la complejidad de interacción. Este crecimiento cuadrático $O(N^2)$ consume el tiempo productivo de los desarrolladores en reuniones, aclaraciones de diseño, sincronizaciones y resolución de conflictos de código.

1.2. La Ley de Brooks y el Cono de Incertidumbre

Fred Brooks (1975) formuló su célebre ley: «Agregar recursos humanos a un proyecto de software retrasado lo hace más tarde aún». Cuando restan 2 meses para la fecha límite, el proyecto se encuentra en una fase avanzada del Cono de Incertidumbre, donde las definiciones arquitectónicas ya están asentadas y el margen para reestructurar tareas es mínimo. Intentar acelerar el ritmo incorporando personal en esta etapa genera una fricción destructiva que

desplaza la fecha de culminación más lejos de lo previsto.

2. Detalle de los Tres (3) Factores de Sobrecarga Operativa

La duplicación abrupta del equipo desencadena tres fenómenos de fricción que degradan el rendimiento neto del proyecto:

- **1. Sobrecarga de Comunicación e Interacción (Communication Overhead):** A medida que el equipo pasa de 5 a 10 integrantes, el esfuerzo requerido para mantener la alineación técnica (standups, documentación, revisión de Pull Requests, alineación de APIs) crece exponencialmente. La probabilidad de desalineaciones o malentendidos sobre la arquitectura aumenta, provocando reuniones de emergencia que restan tiempo a la escritura de código productivo.
- **2. Costo de Aculturación e Integración (Onboarding Overhead):** Los 5 nuevos desarrolladores no son inmediatamente productivos; requieren semanas para comprender el dominio del negocio, la arquitectura existente, las convenciones de código y los entornos de despliegue. Este proceso de capacitación no ocurre en el vacío: los 5 desarrolladores senior originales (quienes poseen el conocimiento crítico) deben pausar su trabajo para mentorear y responder dudas al nuevo personal, reduciendo a corto plazo la velocidad total de entrega del equipo a un nivel inferior al inicial.
- **3. Fricción de Integración, Conflictos de Código y Control de Calidad (Integration Overhead):** Sincronizar el código producido por 10 personas sobre un repositorio ya retrasado incrementa drásticamente la frecuencia de *merge conflicts*, regresiones y fallos de integración. Además, el esfuerzo de aseguramiento de calidad (QA) y la ejecución de pruebas automatizadas se duplican, creando cuellos de botella en la tubería de despliegue continuo (CI/CD) si la infraestructura no estaba preparada para ese volumen.

3. Propuestas de Alternativas de Gestión

Para responder a la crisis de cronograma sin violar los principios de la ingeniería de software ni incurrir en la Ley de Brooks, se formulan dos alternativas fundamentadas:

3.1. Enfoque Predictivo Tradicional: Reducción Táctica de Alcance y Ruta

Crítica Desde la gestión tradicional (PMBOK), el Triángulo de Hierro establece que si el Tiempo está fijo (2 meses) y los Recursos no pueden aumentarse de forma efectiva, la única variable ajustable es el Alcance (Scope). La estrategia consiste en:

- **Análisis del Camino Crítico (CPM):** Identificar la secuencia rígida de tareas dependientes indispensables para que el sistema sea operativo, eliminando holguras ficticias.
- **Poda de Alcance Secundario (Descoping):** Mover inmediatamente las funcionalidades no críticas (deseadas pero no esenciales) fuera de la fecha de entrega original. Se congela el código baseline y el equipo de 5 desarrolladores se enfoca exclusivamente en estabilizar las tareas de la Ruta Crítica para garantizar una entrega funcional en la fecha límite, programando las características secundarias para una versión posterior.

3.2. Enfoque Ágil / Híbrido: Priorización del MVP y Reestructuración en Células Autónomas

Desde la perspectiva ágil, se aplican dos intervenciones simultáneas para maximizar el valor entregado:

- **Foco Rígido en el Producto Mínimo Viable (MVP / MoSCoW):** Reunirse con el Product Owner y los stakeholders para clasificar el backlog mediante la técnica MoSCoW (Must have, Should have, Could have, Won't have). Se cancela el desarrollo de los ítems 'Should' y 'Could', concentrando la capacidad productiva únicamente en las historias de usuario 'Must' indispensables para la salida a producción.
- **Arquitectura de Células de Trabajo Autónomas (Feature Teams):** Si la incorporación de personal es una orden corporativa ineludible, se debe evitar la integración en un solo grupo monolítico de 10 personas. Se divide el equipo en 2 células independientes de 5 desarrolladores cada una (Two-Pizza Teams), con límites de módulo claramente desacoplados mediante contratos de interfaz (APIs). La célula original continúa el trabajo de la entrega inminente, mientras que la nueva célula asume la deuda técnica o la preparación del siguiente release, aislando al equipo principal de la carga de onboarding.

Conclusión

La orden del Director de duplicar el equipo ante una crisis de tiempo es una manifestación clásica de la falacia de los recursos. La ingeniería de software es una actividad basada en la construcción de conocimiento abstracto, no en la fuerza bruta lineal. El análisis matemático demuestra que el incremento del 350% en los canales de comunicación, sumado al costo de entrenamiento y la fricción de integración, ralentizará el proyecto en el periodo crítico de 2 meses.

Los líderes de proyecto deben actuar con rigor técnico, demostrando a la alta dirección que la solución a un retraso de cronograma requiere proteger la capacidad del equipo existente mediante la negociación del alcance (MVP / Descoping) o la segregación modular del trabajo, preservando la calidad y la sostenibilidad de la entrega.